Лабораторная работа № 1. Задание 1

1. Постановка задачи

Требуется написать параллельную программу для нахождения заданного количества точек множества Мандельброта.

Формат входных данных. Запуск из командной строки: ./mandelbrot nthreads npoints, где nthreads - количество потоков, npoints - количество точек.

Формат выходных данных. Файл mandelbrot.csv с координатами точек множества Мандельброта: строка-заголовок re,im, далее npoints строк вида «действительная часть, мнимая часть».

2. Алгоритм

2.1. Критерий принадлежности множеству

Число $c \in \mathbb{C}$ принадлежит множеству Мандельброта тогда и только тогда, когда последовательность

$$z_{n+1} = z_n^2 + c, \qquad z_0 = 0$$

ограничена. В задании это условие записано как $|z_n| < 2$ для всех $n \in \mathbb{N}$. Строго говоря, на границе нужно нестрогое неравенство $|z_n| \le 2$: например, при $c = -2$ последовательность 0, −2, 2, 2, … ограничена, хотя $|z_2| = 2$. Граница $|z| = 2$ имеет нулевую площадь, поэтому на результат это не влияет.

Проверить условие для всех n на компьютере невозможно, поэтому выполняется не более MAX_ITER = 1000 итераций:

- если на некоторой итерации $|z_n| \ge 2$, точка не принадлежит множеству (с точностью до границы, см. выше);
- если за 1000 итераций модуль не достиг 2, точка считается принадлежащей множеству.

Для $z = x + \mathrm{i}y$ и $c = c_\mathrm{r} + \mathrm{i}\cdot c_\mathrm{i}$ одна итерация вычисляется в действительной арифметике:

$$Rez^2 + c = x^2 - y^2 + c_r, \qquad Imz^2 + c = 2xy + c_i$$

Чтобы не вычислять квадратный корень, вместо $|z| \geq 2$ проверяется равносильное условие $x^2 + y^2 \geq 4$.

2.2. Генерация точек

Множество Мандельброта целиком лежит в прямоугольнике $[-2; 0{,}5] \times [-1{,}25; 1{,}25]$ площадью 6,25. В этом прямоугольнике равномерно генерируются случайные точки *c*. Каждая точка проверяется, подходящие сохраняются, пока не будет найдено npoints точек (метод Монте-Карло). В отличие от построчного обхода сетки, точки при этом распределены по всему множеству. В среднем подходит примерно каждая четвертая точка (≈ 24 %).

3. Распараллеливание

3.1. Декомпозиция

Используется блочная декомпозиция. Массив результатов points из *n* элементов делится на *p* непрерывных блоков (*p* - число потоков). Поток с рангом my_rank заполняет элементы с индексами от my_first_i до my_last_i. Если *n* не делится на *p* нацело, первые *n* mod *p* потоков получают на одну точку больше, так что размеры блоков отличаются не более чем на 1. Например, при $n = 10$ и $p = 3$ потоки получают блоки [0..3], [4..6] и [7..9]. Каждый поток вычисляет границы своего блока самостоятельно по своему рангу.

3.2. Отсутствие синхронизации

Каждый поток записывает только в свой блок массива, блоки не пересекаются, поэтому состояние гонки невозможно. Мьютексы, семафоры и барьеры не используются, накладные расходы на синхронизацию отсутствуют. Единственная точка ожидания - вызовы pthread_join в главном потоке. Общие переменные thread_count и n_points записываются до создания потоков и далее только читаются.

3.3. Потокобезопасный генератор случайных чисел

Стандартная функция rand() хранит состояние в скрытой глобальной переменной и не является потокобезопасной: одновременные вызовы из нескольких потоков приводят к гонке, а защита блокировкой выстроила бы потоки в очередь. Поэтому используется генератор xorshift64* (функция Rand_unit), состояние которого хранится в локальной переменной каждого потока. Начальное состояние вычисляется по рангу потока, иначе потоки нашли бы одинаковые точки. Начальные состояния фиксированы, поэтому при одинаковых аргументах программа выдает одинаковый результат.

3.4. False sharing

Счетчик проверенных точек каждый поток ведет в локальной переменной my_tested и записывает в общий массив tested[my_rank] один раз, по завершении работы. Элементы массива tested соседних потоков лежат в одной кэш-строке (128 байт). Если бы поток увеличивал tested[my_rank] на каждой итерации, возник бы false sharing: каждая запись помечала бы кэш-строку устаревшей у остальных ядер. Блоки массива points велики, и общая кэш-строка возможна только на стыке соседних блоков, поэтому их влияние пренебрежимо мало.

3.5. Балансировка нагрузки

Каждый поток должен найти одинаковое число точек my_count, и каждая найденная точка стоит ровно MAX_ITER = 1000 итераций. Случайна только стоимость промахов: на одну найденную точку в среднем приходится $\approx 3{,}1$ промаха (при $n = 10^6$ программа проверяет 4 135 194 точки), а промах прерывается, как только $|z| \geq 2$, и стоит намного меньше 1000 итераций. Поэтому объем вычислений у потоков почти одинаков. Статическое разбиение не учитывает различие в производительности ядер процессора (см. раздел 7).

4. Описание программы

Программа состоит из файла mandelbrot.c, заголовочного файла timer.h (макрос GET_TIME) и файла сборки Makefile.

Таблица 1 - Глобальные переменные и функции программы

Элемент	Назначение
thread_count, n_points	число потоков и требуемое число точек
points	массив найденных точек (общий, каждый поток пишет в свой блок)
tested	число проверенных точек по потокам (для статистики)
main	разбор аргументов, выделение памяти, создание потоков (pthread_create с передачей ранга), ожидание (pthread_join), замер времени, запись результата, освобождение памяти
Thread_work	функция потока: вычисляет границы своего блока и генерирует точки, пока блок не заполнен
In_mandelbrot	проверка принадлежности точки множеству (не более MAX_ITER итераций)
Rand_unit	генератор xorshift64*, возвращает число из [0; 1)
Write_csv	последовательная запись результата в mandelbrot.csv
Usage	вывод формата запуска при неверных аргументах

Сборка и запуск:

```
make
./mandelbrot 4 1000000
```

5. Методика замеров

- Время измеряется макросом GET_TIME из timer.h (функция gettimeofday). В замер входит только параллельная часть: от создания первого потока до завершения последнего (pthread_join). Выделение памяти и запись CSV-файла в замер не входят: запись выполняется последовательно и одинаково при любом числе потоков.
- Для каждого числа потоков p программа запускалась **10 раз** (скрипт bench.sh). Фиксировались минимальное, среднее и максимальное время.
- Ускорение и эффективность вычислялись по **минимальному** времени, так как оно меньше всего зависит от фоновой нагрузки операционной системы:

$$S = \frac{T_1}{T_p}, \qquad E = \frac{S}{p},$$

где в качестве времени последовательной программы взято время работы на одном потоке T_1.

- Количество точек $n = 10^6$ (на одном потоке около 1,7 с).

Аппаратная платформа: Apple M5, 10 ядер - 4 производительных (P) и 6 энергоэффективных (E), общая память; операционная система macOS (Darwin 25.6); компилятор Apple clang 17.0.0, флаги -O2 -Wall -Wextra -std=c11.

Команда запуска замеров: ./bench.sh 1000000 10, где 1000000 - число точек, 10 - число запусков для каждого p.

6. Результаты

Таблица 2 - Время работы, ускорение и эффективность ($n = 10^6$, 10 запусков)

p	T_{min}, с	T_{avg}, с	T_{max}, с	S	E
1	1,7257	1,7362	1,7963	1,00	1,00

p	T_{min}, с	T_{avg}, с	T_{max}, с	S	E
2	0,8726	0,8835	0,9533	1,98	0,99
4	0,4534	0,4674	0,5671	3,81	0,95
6	0,3509	0,3609	0,3794	4,92	0,82
8	0,2910	0,3044	0,3274	5,93	0,74
10	0,2531	0,2715	0,3040	6,82	0,68
12	0,2512	0,2579	0,2610	6,87	0,57
16	0,2460	0,2798	0,4209	7,02	0,44

Проверка корректности. По доле попаданий можно оценить площадь множества: $6{,}25 \cdot 10^6 / 4\ 135\ 194 \approx 1{,}511$, где 6,25 - площадь прямоугольника, а 4 135 194 - число точек, проверенных при $p = 1$, чтобы найти n точек (значение «проверено» из вывода программы). Это близко к известному значению ≈ 1,507. Небольшое завышение объясняется конечным значением MAX_ITER: часть точек, не ушедших за 1000 итераций, ушла бы позже. Остальное - статистическая погрешность метода Монте-Карло.

7. Анализ результатов

1. **До 4 потоков ускорение почти линейное** ($E \geq 0{,}95$). Задача является естественно параллельной: потоки не обмениваются данными и не синхронизируются, а последовательная часть (создание и ожидание потоков) пренебрежимо мала по сравнению с вычислениями. Экспериментальную долю последовательных вычислений r можно оценить по формуле Карпа-Флэтта:

$$r = \frac{\frac{1}{S} - \frac{1}{p}}{1 - \frac{1}{p}}.$$

При $p = 2$ получается $r \approx 0{,}010$, при $p = 4$ $r \approx 0{,}017$. По закону Амдала ускорение не может превысить $1/r \approx$ 60-100, что намного

больше числа ядер, поэтому последовательная часть не ограничивает ускорение на данной машине.

2. **После 4 потоков эффективность снижается** из-за гетерогенности процессора: 6 из 10 ядер - энергоэффективные (E), они медленнее производительных (P). Привязка потоков к ядрам в программе не задается, распределение выбирает ОС. Переход от 4 к 6 потокам увеличил ускорение на ≈ 1,1, то есть каждое из двух добавленных E-ядер дало ≈ 0,55 от производительности P-ядра. Это эффективная величина: она учитывает и то, что ОС переносит недоделанные потоки с E-ядер на освободившиеся P-ядра (без такого переноса при $p = 6$ время определял бы самый медленный поток на E-ядре, и ускорение было бы ниже 4). Отсюда грубая оценка предельного ускорения на данной машине: $3{,}81 + 6 \cdot 0{,}55 \approx 7{,}1$, а не 10.
3. **Балансировка нагрузки.** Разбиение статическое: все потоки получают одинаковое число точек, а время работы программы определяется самым медленным потоком. При $p = 10$ потоки на P-ядрах завершаются раньше; оставшиеся потоки ОС частично переносит на освободившиеся P-ядра, но часть времени P-ядра простаивают. При $p > 10$ ускорение выходит на плато ≈ 6,9-7,0, что согласуется с оценкой выше. Различие между $p = 12$ и $p = 16$ (6,87 и 7,02) находится в пределах разброса замеров, а по среднему времени $p = 16$ даже хуже, чем $p = 12$ (0,280 с против 0,258 с). При $p = 16$ заметно возрастает разброс времени ($T_{max} = 0{,}421$ с при $T_{min} = 0{,}246$ с).
4. **Масштабируемость.** На производительных ядрах ($p \leq 4$) эффективность не ниже 0,95, что указывает на хорошую сильную масштабируемость (рост числа потоков при фиксированном n). На гетерогенном процессоре эффективность ограничена различной производительностью ядер, а не накладными расходами самой

программы. Слабая масштабируемость (рост n вместе с p) не исследовалась.